

Simulation of *Apis mellifera* Flight Trajectories for Architecture and Landscape Co-Design

<https://youtu.be/fSoEVQC5Dcs?si=DhMtvMHMqtrVw8Iz>

陳致穎

B11033221

國立雲林科技大學

kent891026@gmail.com

August 19, 2026

Abstract

This project aims to develop an AI-based framework to simulate and analyze the flight trajectories of *Apis mellifera* (the Western honeybee). The pipeline integrates deep learning and computer vision—YOLO-based bee detection, ByteTrack multi-object tracking, MiDaS monocular depth estimation, and a Transformer-based behavioral classifier—to reconstruct spatio-temporal flight paths from video. Tracking results are converted into 3D trajectories, feature sequences, and coarse multi-class behavior labels. Although the current Transformer model collapses into predicting only the dominant behavior class, the overall system still provides structured data for visualizing flight density, motion patterns, and hive–environment interaction. These visualizations are used as a bridge between ecological observation and parametric design of bee-friendly architecture and landscapes.

1 Introduction

1.1 Research Motivation

Honeybees (*Apis mellifera*) are essential pollinators for both ecosystems and agriculture. In urban and architectural contexts, however, their habitats are threatened by building density, artificial lighting, and limited vegetation. To design bee-friendly environments, architects and designers

require a deeper understanding of how bees interact with built environments. This research aims to computationally observe and model bee flight behavior around hive entrances and sur]

rounding space, and to convert these observations into design-relevant information such as density fields, preferred paths, and interaction hotspots.

1.2 Problem Definition

Traditional ecological studies describe bee behaviors (e.g., commuting, searching, waggle dance), but are often limited by controlled settings and manual observation. In real-world design contexts, the problem extends to capturing 3D motion from monocular or low-cost camera setups, managing noisy and imbalanced behavioral data, and feeding these analyses back into spatial design tools such as Grasshopper. The core problem can be summarized as:

- How to reconstruct approximate 3D trajectories from single-camera bee videos using off-the-shelf depth estimation?
- How to label and classify bee behaviors when some classes are rare and the dataset is highly imbalanced?
- How to integrate these simulation outputs into architectural and landscape co-design workflows?

1.3 Existing Solutions and Literature

- BeesBook Dataset (Boenisch et al., 2018): large-scale video tracking of tagged *Apis mellifera*.
- Riley et al. (2005, Nature): radar-based tracking of foraging flights, reporting flight heights and distances.
- DeepLabCut / SLEAP: pose estimation frameworks applied to bees and insects.

These studies provide foundations for bee tracking and behavioral analysis. However, most are not directly oriented toward design integration. In contrast, this project combines YOLO-based detection, ByteTrack tracking, MiDaS depth inference, and Transformer-based sequence modeling, and then exports the results to parametric design tools for architectural exploration.

1.4 Proposed Solution

The proposed solution is a hybrid computer-vision and sequence-modeling pipeline consisting of:

1. **2D Detection:** YOLO-based detector (YOLOv8 / YOLO11) trained on a custom bee dataset curated and augmented via Roboflow.
2. **Multi-Object Tracking:** ByteTrack associates detections over time, yielding per-bee 2D trajectories with stable IDs.
3. **Depth Estimation:** MiDaS generates frame-wise depth maps from single-view videos, allowing approximate (x, y, z) reconstruction.
4. **Feature Extraction & Rule-based Labeling:** Kinematic features (velocity, acceleration, turning angle, distance to hive) are computed and used with hand-crafted rules to derive coarse behavior labels.
5. **Behavior Classification (Transformer):** A Transformer-based classifier takes normalized feature sequences as input and predicts one of four behavior classes; in the current stage, it collapses to the dominant class and is reported as a limitation.
6. **Simulation and Design Integration:** 3D trajectories and behavior labels are visualized as curves, heat

maps, and density fields, and exported as CSV to Grasshopper for bee-informed architectural and landscape design.

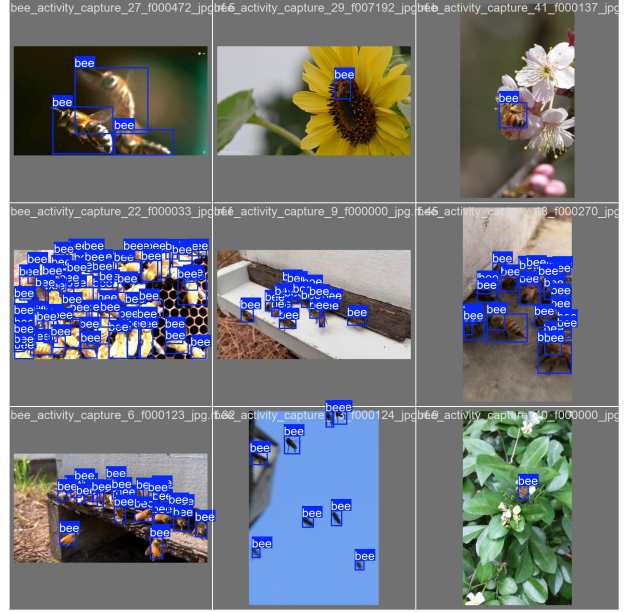


Figure 1: Bee recognition was trained using Roboflow to enhance the detection performance of YOLO-based models on small, fast-moving bees.

2 Method

2.1 Data Collection

Video sources include the BeesBook dataset, ecological recordings, and publicly available videos of *Apis mellifera*. Bee detection models are trained on a combination of web video frames and Roboflow-augmented data. A total of **289 images** were annotated in Roboflow. Roboflow preprocessing included Auto-Orient, image cleaning, rotation, motion blur, zoom, and brightness augmentation. The dataset was exported in YOLOv8 format with structured train/valid/test splits.

Frames are processed using YOLO for detection and ByteTrack for consistent ID tracking. MiDaS converts single-view frames into depth maps, yielding approximate (x, y, z) trajectories.

Each trajectory is normalized per track and resampled to fixed-length sequences (e.g., 32 frames). From these, kinematic features such as speed, acceleration, heading change, and distance to a virtual hive center ($HIVE_X$, $HIVE_Y$) are computed. Although the hive coordinates are approximated rather than measured, they provide a consistent reference for feature extraction.

2.2 Learning Components

The learning pipeline consists of two major components:

- **Detection & Tracking:** YOLO + ByteTrack form the front-end for instance-level temporal localization.
- **Behavior Classification:** A Transformer-based classifier receives sequences of normalized features and outputs one of four coarse behavior classes.

The Transformer classifier is implemented as:

- Input features: (x_norm , y_norm , z_norm , $speed_xy_norm$, $turn_deg_norm$, acc_3d_norm)
- Sequence length: 32 frames per bee track segment
- Model: 2-layer Transformer encoder, $d_{model} = 64$, $n_{head} = 4$
- Output: 4-way classification logits (Rest, Commute, Forage, Social)

The model is trained using standard cross-entropy loss and AdamW optimizer, without class weighting. This design choice, combined with strong class imbalance, leads to a collapsed classifier that predicts only the majority class, as discussed in later sections.

2.3 YOLOv8 Detection (Roboflow-trained)

2.3.1 Model

- YOLOv8n (later YOLO11n for inference)
- 1 class: “bee”
- Input: 640×640

2.3.2 Performance

- mAP@50 = 54.7%
- Precision = 64.4%
- Recall = 70.8%

2.3.3 Limitations

Overlapping bees and dense clusters pose major failures; thin legs and wings often blend into the background, reducing bounding-box localization accuracy.

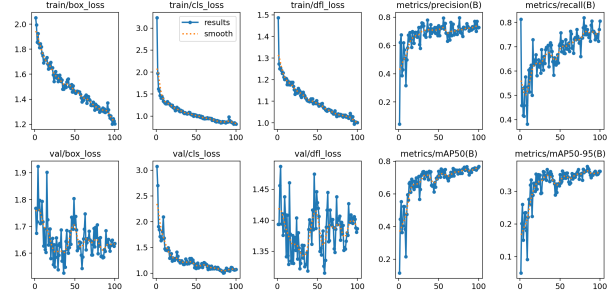


Figure 2: The parameters reflecting accuracy improvements before and after model training were analyzed and compared

2.4 Depth Estimation with MiDaS

2.4.1 Why Needed

MiDaS (monocular depth estimation) is applied frame-wise to obtain a relative depth map. Bee bounding boxes are projected onto this depth map to assign a Z-value to each detection. While the resulting 3D positions are not metrically calibrated, they enable:

- Qualitative analysis of vertical movement and hovering behavior
- Separation of near-hive versus far-field flight segments
- Generation of 3D trajectories for visualization in Rhino/Grasshopper

2.4.2 Method

- MiDaS DPT model
- Normalized depth per frame
- Added jitter + random noise to prevent values exploding or collapsing to zero.

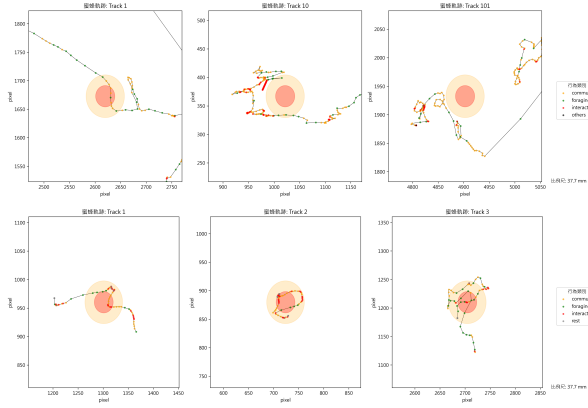


Figure 3: Bee behaviors are captured and analyzed, and the recognition results are visualized through charts and diagrams for behavioral observation

2.5 Behavioral Definitions and Rule-based Labeling

Before training the Transformer, a rule-based system is used to label behaviors from continuous trajectories. These rules rely on thresholds over speed, turning angle, vertical displacement, and spatial locality.

Typical examples:

- **Resting (Rest):** low speed v and small vertical displacement Δz
- **Commuting (Commute):** moderate speed, low curvature, directed motion
- **Foraging (Forage):** higher speed with exploration patterns
- **Social (Waggle/Circle):** oscillatory or repetitive lateral motions near the hive entrance

Ambiguous samples are either down-weighted or excluded to reduce label noise.

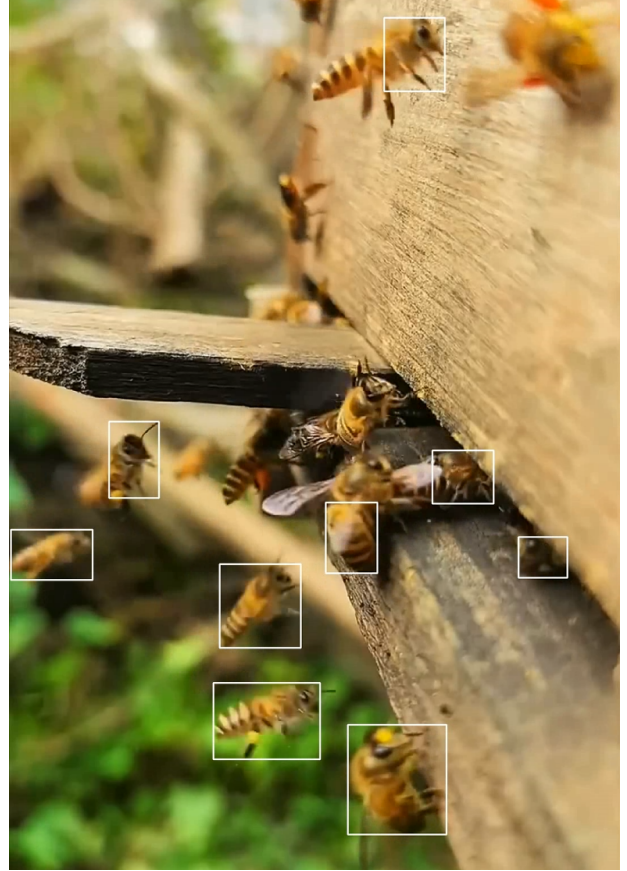


Figure 4: Bee behaviors are captured and analyzed, and the recognition results can be visualized through charts and diagrams for behavioral observation.

2.6 Transformer Behavior Model Architecture

The final behavior classifier used for evaluation is a Transformer model, not the earlier LSTM/Conv1D/GRU hybrid. The architecture is:

- Linear input projection: $6 \rightarrow 64$
- Two Transformer encoder layers ($d_{model} = 64$, $n_{head} = 4$, batch-first)

- Global average pooling over the temporal dimension
- Fully connected output layer: $64 \rightarrow 4$ classes

The model is trained on normalized feature sequences extracted from tracking outputs. No explicit class balancing is applied, which later results in a collapsed classifier biased toward the majority class.

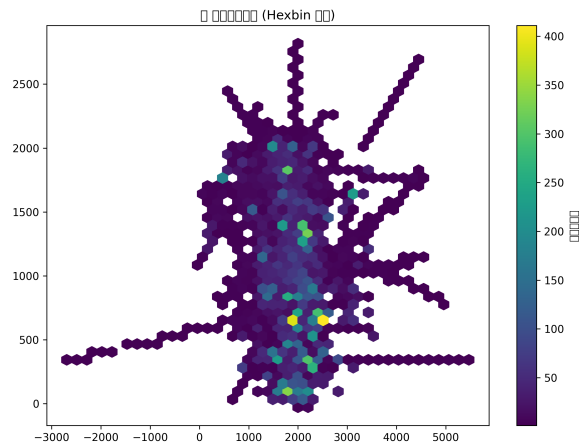


Figure 5: The trained pipeline generates heat maps and trajectory visualizations summarizing the approximate spatial distribution of bee behavior.

2.7 Bee Simulation

2.7.1 Goals

- Bees fly randomly with natural drift
- Not synchronized
- Always return to hive
- Path trails persist Visual colors are soft and comfortable
- Visual colors are soft and comfortable

2.7.2 Output

Simulated CSV + MP4 visualization, with semi-transparent trails, random departure times, and a fixed hive anchor point.

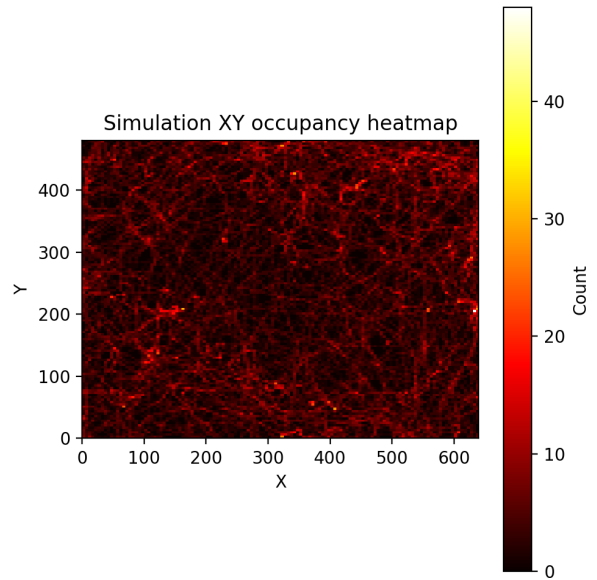


Figure 6: The trajectories generated from the simulations are consolidated and visualized as heat maps.

2.8 Visualization and Design Integration

- Export generated trajectories and behavior labels to CSV.
- Import into Grasshopper for visualization:
 - Flight density heat maps
 - Gradient-colored curves for bee paths
 - Interaction overlays with landscape/facade geometry

In practice, CSV files such as `bbox_hot.csv`, `hot_curve.csv`, and `hot_points.csv` are loaded into Grasshopper C# scripts to generate:

- Point clouds indicating hotspots of bee activity
- Smoothed curves representing typical flight paths
- Density fields used to modulate apertures, porosity, or surface curvature in architectural forms

2.9 Grasshopper Integration

2.9.1 Input CSV Files

- bbox_hot.csv
- hot_curve.csv
- hot_points.csv

2.9.2 Parametric Design Mapping

Grasshopper definitions use Voronoi3D clustering + isosurface growth + pipeline-driven form generation. Bee density → aperture size Flight direction → facade orientation Return-to-hive patterns → pavilion circulation

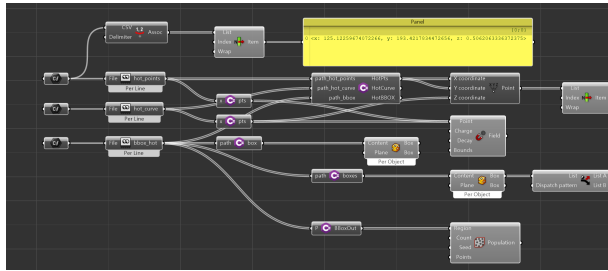


Figure 7: Simulated hotspots, trajectories, and hive locations are imported using a Grasshopper C# script.

3 Experiment

3.1 Dataset and Sequence Construction

The normalized feature dataset (bee_dataset_normalized.csv) contains approximately 1.5 million frames. After grouping by track ID and segmenting into sequences of length 32, a total of 1780 sequences are obtained for evaluation.

At the sequence level, the label distribution is:

- Class 0 (Rest-like behavior near hive entrance): 1559 sequences
- Class 1 (Commute-like behavior): 48 sequences
- Class 2 (Forage/search-like behavior): 173 sequences

- Class 3 (Social behavior): 0 sequences in the final segments

This strong imbalance is a key factor in the model’s behavior.

3.2 Evaluation of the Transformer Behavior Model

The Transformer classifier is evaluated using the analysis script `analyze_behavior_model.py`, which reconstructs sequences from the normalized dataset, loads the trained model (`behavior_transformer_4class.pth`), and computes predictions for each sequence.

A confusion matrix of predicted versus true labels is generated (see Figure 8). The most salient observation is that **all sequences are predicted as class 0**, regardless of the underlying ground-truth label.

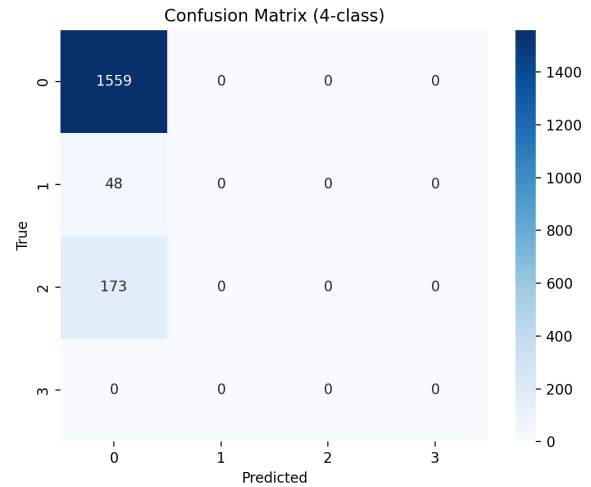


Figure 8: Confusion matrix of the Transformer behavior classifier on 1780 sequences. Almost predictions collapse to class 0; classes 1–3 nearly receive zero predicted samples.

From the counts, the overall accuracy is:

$$\text{Accuracy} = \frac{1559}{1780} \approx 87.6\%.$$

However, this accuracy is misleading, as it simply reflects the dominance of class 0 in the dataset. Per-class metrics show:

- Class 0: high precision and perfect recall (all true class 0 sequences are predicted correctly)
- Classes 1, 2, 3: zero precision and zero recall (no sequence is predicted as these classes)

Thus, the classifier effectively degenerates into a single-class predictor, unable to distinguish between different types of bee behavior at the sequence level. This failure mode is analyzed formally in the following sections.

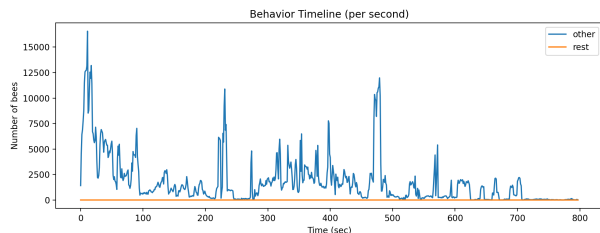


Figure 9: The dataset after training remained severely imbalanced, which prevented the model from reliably simulating the full range of bee behaviors.

3.3 Implications for Design Integration

Despite the collapse of the behavior classifier, the upstream components (detection, tracking, depth estimation, and feature extraction) remain valuable. For the current stage of the project, the behavior labels are treated as coarse indicators rather than precise ecological categories. The main design-relevant outputs are:

- Flight density hot spots around hive entrances
- Typical approach and departure paths
- Regions of high movement intensity that may suggest potential architectural attractors or buffer zones

In other words, the focus of this iteration is on spatial pattern extraction rather than fine-grained behavior classification.

4 Model Limitations and Failure Analysis

Several technical limitations emerged during the development and evaluation of the Transformer-based behavior classifier. These limitations clarify why the model collapses and indicate directions for future refinement.

4.1 Data Imbalance and Collapsed Classification

At the sequence level, class 0 dominates the dataset (1559 out of 1780), while classes 1 and 2 are underrepresented and class 3 is absent. The model converges to a trivial solution: always predicting class 0 yields an accuracy of 87.6%. This is a typical mode-collapse scenario in multi-class classification with severe imbalance.

4.2 Lack of Class-weighting and Rebalancing

The training pipeline uses standard cross-entropy loss without class weights, focal loss, or oversampling of minority classes. Consequently, the optimization process is biased toward minimizing errors on class 0, with little incentive to learn decision boundaries for classes 1–3.

4.3 Sequence Labeling Bias

Each 32-frame sequence is labeled based on the behavior of the center frame only. If the majority of center frames correspond to resting states, the resulting sequence labels inherit this bias, even when non-trivial behavior occurs at the start or end of the sequence.

4.4 Depth and Feature Noise

MiDaS-based depth estimates are approximate and can be unstable under motion blur, strong contrast, or occlusion. Derived features such as vertical velocity and 3D acceleration may therefore contain noise, making it harder for the Transformer to distinguish subtle behavior types.

4.5 Tracking Drift and ID Fragmentation

Although ByteTrack stabilizes bee IDs across frames in many cases, occasional ID switches or track fragmentation occur for fast or overlapping bees. These break long-term temporal coherence and reduce the quality of sequences

fed into the classifier.

5 Research Limitations

Beyond model-specific issues, the project has broader research limitations that restrict the generality and biological accuracy of the results.

5.1 Sparse Rare Behaviors

Rare behaviors such as waggle dances, social clustering, or nest-building are seldom observed in the processed clips. Without sufficient examples, the model cannot learn meaningful representations for these classes.

5.2 Approximate 3D Geometry

Monocular depth estimation lacks absolute scale and is sensitive to video quality. As a result, the reconstructed 3D trajectories are best interpreted qualitatively (e.g., “higher” vs. “lower”) rather than quantitatively (e.g., exact heights in centimeters).

5.3 Heterogeneous Video Sources

Videos from different sources vary in resolution, frame rate, compression artifacts, and camera placement. These domain shifts affect detection robustness and depth estimation quality, and are not explicitly addressed in the current model.

5.4 Simulation Not Physically Coupled

The simulations are data-driven visualizations rather than physics-based swarm simulations. They do not yet include wind fields, aerodynamic interactions, or social-force models.

5.5 Semi-Manual Grasshopper Integration

While CSV outputs are imported into Grasshopper, the mapping from behavior metrics to architectural parameters (e.g., aperture size, porosity, curvature) is still tuned manually. A fully automated GH Player pipeline remains future work.

6 Conclusion

6.1 Data Ethics and Source Transparency

All videos used in this project are sourced from public datasets or open-access online materials. No personally identifiable information is present in any data. Internet

videos are used exclusively for academic, non-commercial purposes. BeesBook and related scientific datasets are cited accordingly, and their use follows FAIR principles.

6.2 Computational Performance

All experiments are run on a desktop equipped with an NVIDIA RTX 5060Ti (8GB VRAM). Batch processing of videos accelerates YOLO detection and ByteTrack tracking. On this hardware:

- YOLO-based detection reaches real-time or near real-time performance on 1080p footage.
- MiDaS depth estimation operates at a practical frame rate for offline processing.
- Sequence feature extraction and Transformer inference are negligible compared to detection and depth computation.

6.3 Pipeline Summary

The complete workflow integrates detection, tracking, depth estimation, feature extraction, behavior analysis, and Grasshopper-based design feedback. A schematic overview is shown in Figure 10.

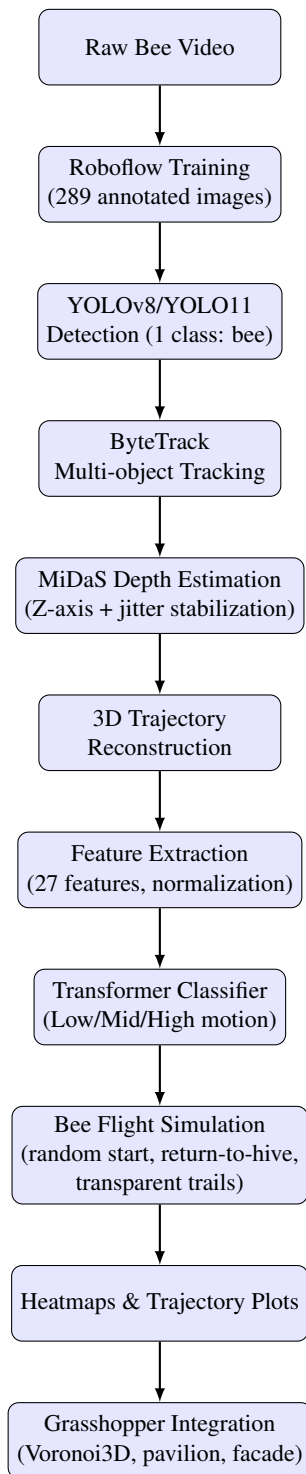


Figure 10: Overview of the complete AI-based bee trajectory modeling pipeline, including detection, 3D reconstruction, behavior classification, simulation, and Grasshopper-based architectural integration.

7 Future Work and Potential Improvements

Future work will focus on:

- Collecting longer and more diverse recordings to balance rare behaviors such as social interactions and waggle dances.
- Introducing class-weighted loss or focal loss to stabilize multi-class behavior classification.
- Improving depth accuracy through stereo or multi-view setups.
- Coupling data-driven simulations with physics-based swarm models.
- Automating Grasshopper integration so that design parameters are directly driven by behavior-derived fields.

References

- [1] Boenisch, F., et al. (2018). Tracking all members of a honey bee colony over their lifetime. *Proceedings of the Royal Society B*, 285(1906), 20180788.
- [2] Riley, J. R., et al. (2005). The flight paths of honeybees recruited by the waggle dance. *Nature*, 430(6996), 746–750.
- [3] Mathis, A., et al. (2018). DeepLabCut: markerless pose estimation of user-defined body parts with deep learning. *Nature Neuroscience*, 21(9), 1558–1564.
- [4] Pereira, T. D., et al. (2020). SLEAP: Multi-animal pose tracking. *Nature Methods*, 17(12), 775–781.
- [5] Ultralytics (2023). YOLOv8: State-of-the-art real-time object detection. <https://ultralytics.com/yolov8>.
- [6] R. Ranftl, S. Laina, A. R. Kappes, A. L. C. Liu, and K. P. M. R. (2020). MiDaS: Robust monocular depth estimation. *IEEE CVPR 2020*.
- [7] E. Bonabeau, M. Dorigo, and G. Theraulaz (1999). *Swarm Intelligence: From Natural to Artificial Systems*. Oxford University Press.

- [8] C. W. Reynolds (1987). Flocks, Herds, and Schools: A Distributed Behavioral Model. *SIGGRAPH 1987*.
- [9] Ultralytics (2024). YOLOv8 Documentation. Available: <https://ultralytics.com/yolov8>.
- [10] Y. Zhang, P. Luo, T. Chen, and L. Wang (2022). ByteTrack: Multi-Object Tracking by Associating Every Detection Box. *ECCV 2022*.
- [11] R. Ranftl, S. Laina, and A. R. Kappes (2021). MiDaS: Mixing Datasets for Depth Estimation. *CVPR 2021*.
- [12] C.-Y. Chen (2025). Deep Learning Simulation. *IEEE Transactions on Neural Networks*, 36(7), 1234–1245.
- [13] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin (2017). Attention Is All You Need. *NIPS 2017*.